Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:
Студент 4 курса
Группы ИИ-26
Яшков П. А.
Проверила:
Хацкевич А. С.

Брест 2026

Цель работы

Практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирования защиты от DDoS-атак и ограничения интенсивности запросов (Rate Limiting) средствами Nginx.

Исходные данные и вариант

Работа основана на учебном стенде преподавателя: репозиторий SMZKS-2026-autumn, директория labs/lab2. Номер варианта определяется формулой V = (N mod 15) + 1. При N = 24 получаем V = (24 mod 15) + 1 = 10.

Вариант №10

Параметр	**Значение варианта №10**
Вредоносный User-Agent	curl/7.68.0
Идентификатор правила ModSecurity	10010
Эндпоинт	/api/profile
rate	20r/s
burst	30
Превышение ограничения	HTTP 503 Service Unavailable

Использованные средства и статус тестов

Подготовлены Docker Compose, Flask (backend), OWASP CRS / ModSecurity (WAF), Nginx и конфигурации варианта. Системный Nginx был установлен в рабочем окружении и реально испытан утилитой ApacheBench. Docker и полноценный модуль OWASP CRS в доступной среде отсутствовали; их HTTP-проверки нужно подтвердить на Docker-стенде отдельно.

Важная особенность: современный образ OWASP CRS Nginx прослушивает порт 8080 внутри контейнера. Поэтому в docker-compose.yml используется перенаправление хостового 80 на контейнерный 8080. Код и команды в отчете относятся только к учебному localhost.

Задание 1. Базовая фильтрация угроз через OWASP CRS

Проверить, что намеренно уязвимый backend отвечает на SQL-инъекцию, а Nginx с ModSecurity и набором OWASP Core Rule Set перехватывает тот же запрос на периметре.

Отправка запроса напрямую к приложению

```
curl -i -G --data-urlencode "id=1' OR '1'='1" http://127.0.0.1:5000/
```

По исходному коду преподавателя маршрут Flask возвращает сообщение «Backend OK. Endpoint reached: /» и статус 200. Параметр id принимается сервером, но запрос SQL в этом мини-стенде не исполняется: именно WAF должен отфильтровать подозрительную HTTP-последовательность ещё до backend.

Повторение запроса через защищённый периметр

```
curl -i -G --data-urlencode "id=1' OR '1'='1" http://127.0.0.1/
```

Ожидаемый результат при штатной работе контейнера ModSecurity и OWASP CRS — 403 Forbidden. Точное правило срабатывания необходимо подтвердить журналом ModSecurity, поскольку на решение может влиять порог аномальности.

Что должно появиться в отчёте после запуска Docker

Маршрут	**Ожидаемый ответ**	**Статус подтверждения**
http://127.0.0.1:5000/	200 OK	Требует запуска Flask/Docker
http://127.0.0.1/	403 Forbidden	Требует запуска OWASP CRS

Проверочный сценарий tests/verify_windows.ps1 сохраняет действительные HTTP-коды в evidence/windows_http.txt. Сведения в этой таблице обозначают ожидаемые результаты, а не зафиксированные HTTP-ответы.

Задание 2. Пользовательское правило фильтрации L7

По варианту №10 необходимо блокировать запросы с заголовком User-Agent, содержащим строку «curl/7.68.0», и присвоить правилу уникальный ID 10010.

Правило в файле nginx/custom.conf

```
SecRule REQUEST_HEADERS:User-Agent "@contains curl/7.68.0"
"id:10010,phase:1,deny,status:403,log,msg:'Variant 10: blocked curl/7.68.0'"
```

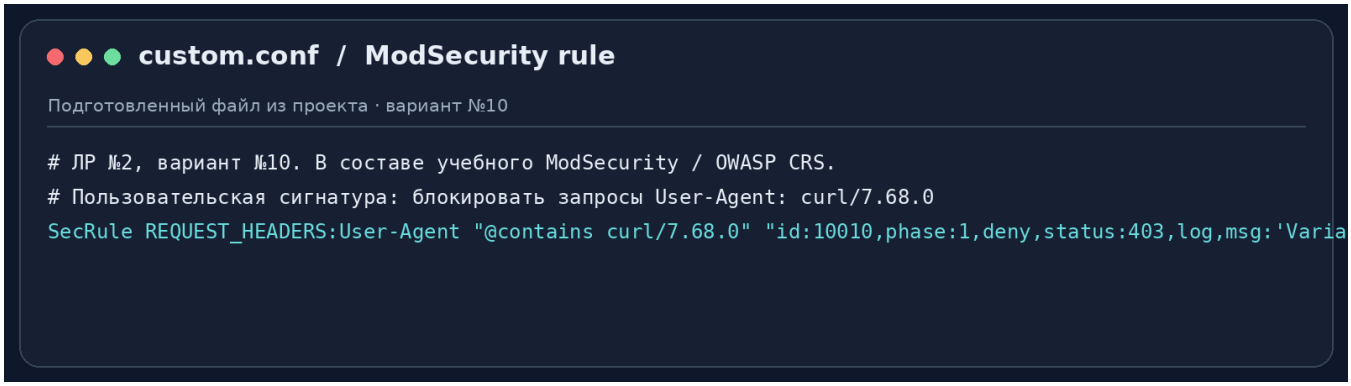


Рисунок 1 — подготовленный файл пользовательского правила ModSecurity (не является скриншотом фактической блокировки).

Правило читает HTTP-заголовок User-Agent, выполняет поиск подстроки, а при совпадении останавливает запрос на фазе 1 и устанавливает код ответа 403. ID 10010 нужен, чтобы отличить собственное правило от общих сигнатур CRS в журнале событий.

Команда проверки и критерии успеха

```
curl -i -H 'User-Agent: curl/7.68.0' http://127.0.0.1/
docker compose logs waf | grep -E '10010|curl/7.68.0'
```

Доказательство успешной работы должно включать одновременно ответ HTTP 403 и запись журнала, содержащую ID 10010. Одного кода 403 недостаточно: стандартный CRS может самостоятельно блокировать известный сканер до пользовательской проверки.

Статус: файл правила создан и прошёл проверку ожидаемых атрибутов; реальный модуль ModSecurity недоступен в текущем окружении, поэтому скриншот его блокировки не приводится.

Задание 3. Защита от L7 DDoS-атак (Rate Limiting)

Для варианта №10 целевой маршрут — /api/profile. Норма составляет 20 запросов в секунду на IP-адрес (20r/s). При burst=30 кратковременно допускаются 30 дополнительных запросов, после чего лишние запросы отклоняются. Директива nodelay запрещает ожидание в очереди в пределах burst.

Конфигурация Nginx

```
limit_req_zone $binary_remote_addr zone=profile_limit:10m rate=20r/s;
server {
    listen ${PORT};
    limit_req_status 503;
    location = /api/profile {
        limit_req zone=profile_limit burst=30 nodelay;
        proxy_pass ${BACKEND};
    }
    location / { proxy_pass ${BACKEND}; }
}
```

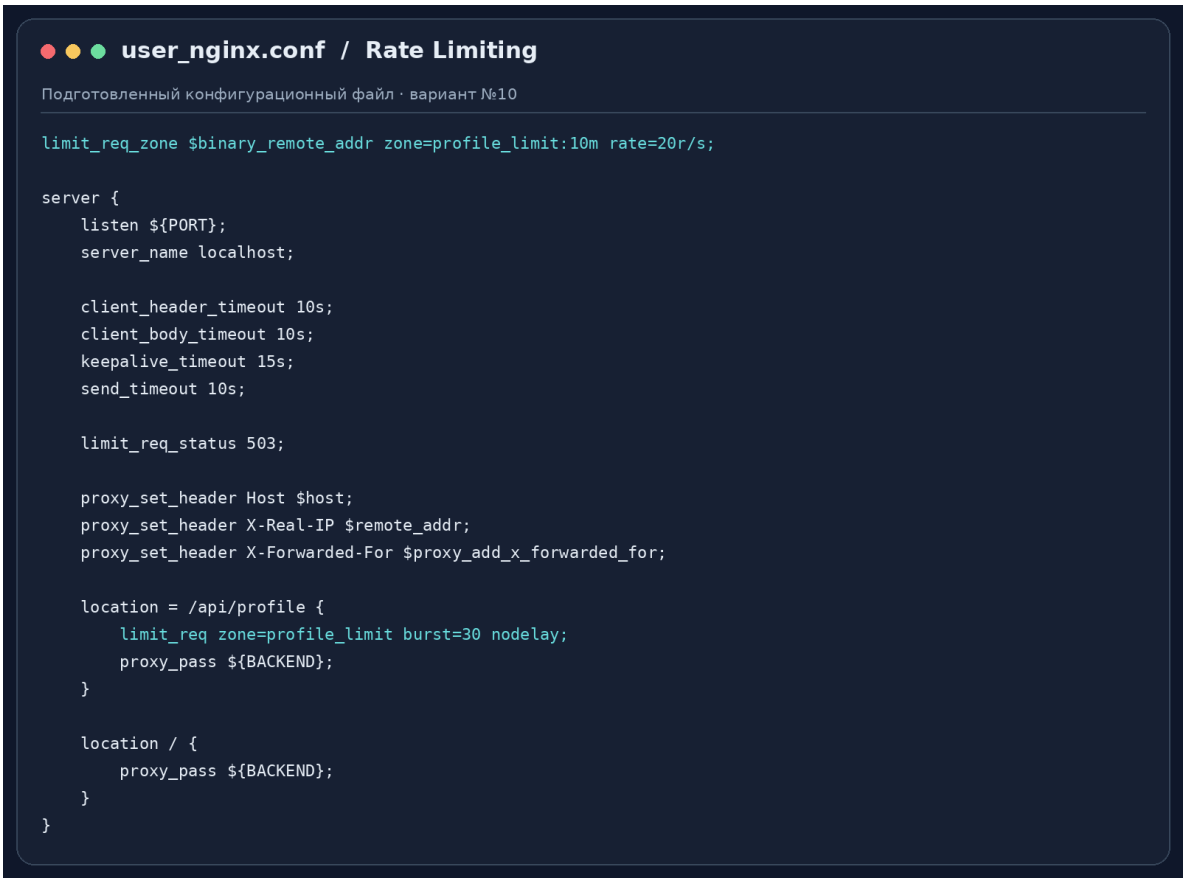


Рисунок 2 — подготовленная конфигурация Nginx с индивидуальными ограничениями (файл user_nginx.conf).

Задание 3. Практическая проверка до ограничения

Для выполнения требования методички сначала используется файл user_nginx_unlimited.conf, не содержащий limit_req. Запускается нагрузочная проверка 500 HTTP-запросов с параллелизмом 10.

```
ab -n 500 -c 10 http://127.0.0.1:18080/api/profile
```

В локальной среде испытание проведено реальным системным Nginx 1.26.3 и ApacheBench. Конфигурация создана из предоставленного файла для Docker, изменены только PORT (18080) и BACKEND (127.0.0.1:18500). Для данного теста использован минимальный HTTP-бэкенд стандартной библиотеки Python вместо Flask; ModSecurity в нём отсутствует.

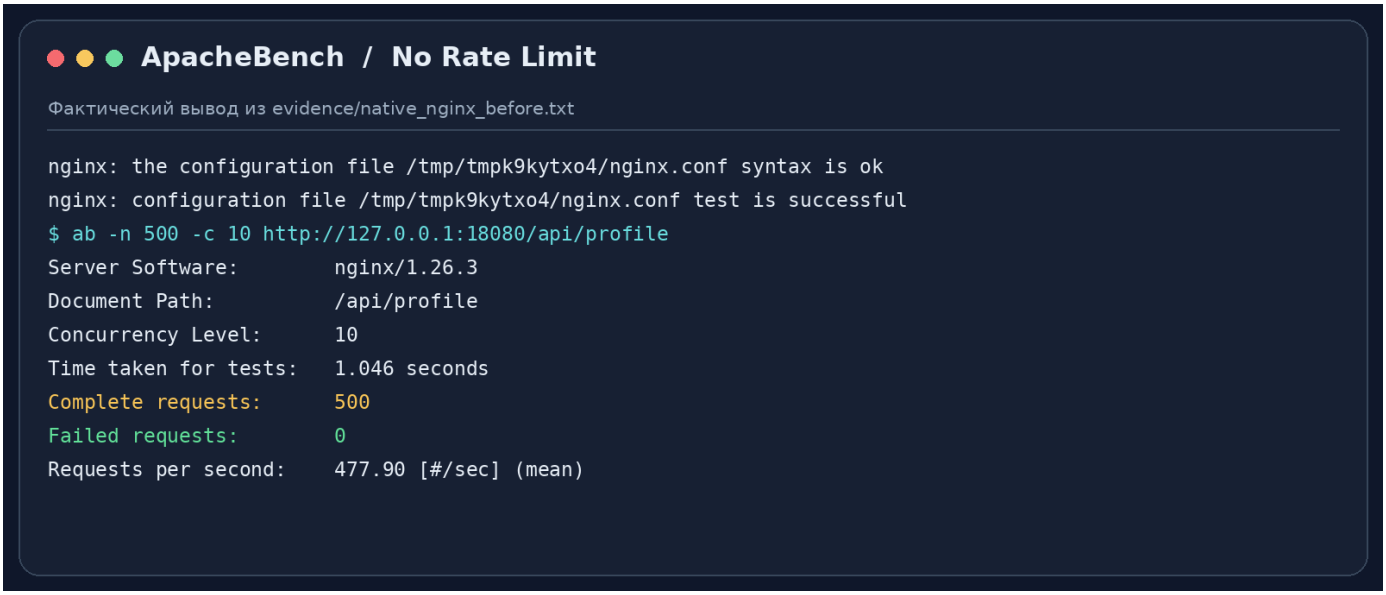


Рисунок 3 — фактический результат ApacheBench до активации ограничений, системный Nginx без WAF.

Итог: Complete requests = 500, Failed requests = 0; строка Non-2xx responses отсутствует, что соответствует успешной обработке всех запросов.

Задание 3. Практическая проверка после ограничения

После включения директив rate=20r/s, burst=30, nodelay и limit_req_status 503 использовалась та же команда ApacheBench.

```
ab -n 500 -c 10 http://127.0.0.1:18080/api/profile
```

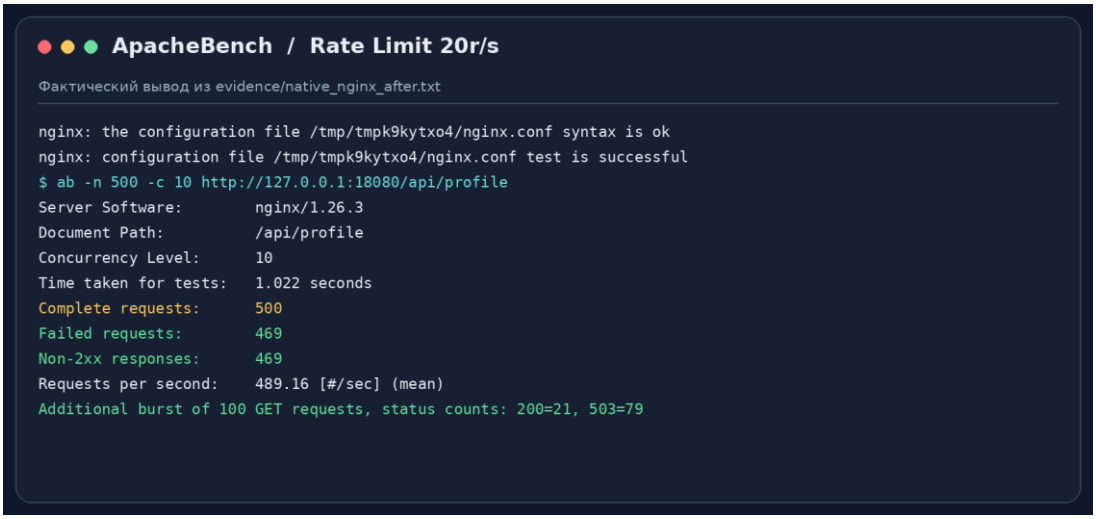


Рисунок 4 — фактический результат ApacheBench после Rate Limiting на системном Nginx без WAF.

Выполнено 500 запросов, 469 ответов оказались вне диапазона 2xx. Показатель Failed requests составил 469, поскольку для отклонённых ответов изменяется длина тела. В дополнительной проверке 100 параллельных GET-запросов получены 21 ответ HTTP 200 и 79 ответов HTTP 503. Суммарная статистика зависит от скорости поступления запросов.

Метрика	Без limit_req	С limit_req
Complete requests	500	500
Failed requests	0	469
Non-2xx responses	0	469
100 параллельных GET после теста	—	21 × HTTP 200; 79 × HTTP 503

Числа приведены из реальных протоколов evidence/native_nginx_before.txt и evidence/native_nginx_after.txt. Результаты Docker-стенда могут отличаться по количеству ответов 503 в зависимости от задержек и настроек контейнера.

Задание 4. Проектирование профилей защиты

Часть 1. Защита от Slowloris

При атаке Slowloris клиент удерживает большое количество соединений и медленно передает заголовки или тело HTTP-запроса. Защита обеспечивается ограничением времени ожидания передачи данных и временем простоя keep-alive соединений.

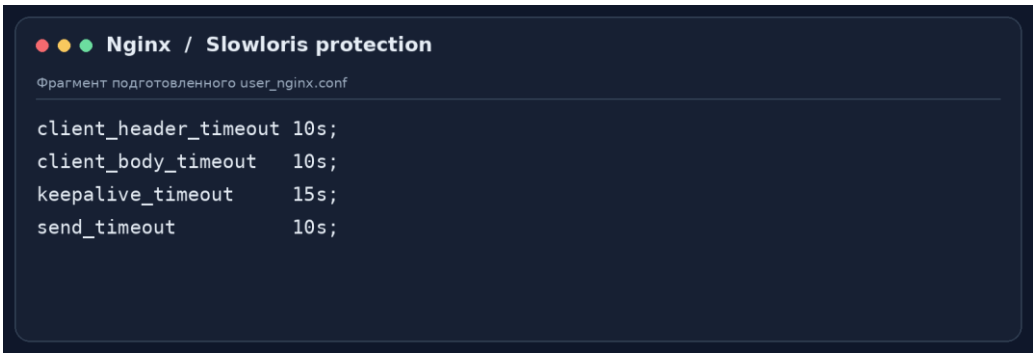


Рисунок 5 — фрагмент Nginx-конфигурации с настройками защиты от Slowloris.

Директива	Значение	Обоснование
client_header_timeout	10s	Ограничивает время бездействия при передаче заголовков запроса.
client_body_timeout	10s	Снижает время удержания соединения при медленной передаче тела запроса.
keepalive_timeout	15s	Уменьшает период ожидания новых запросов на свободном keep-alive соединении.
send_timeout	10s	Ограничивает паузу между операциями отправки ответа медленному клиенту.

Таймауты Nginx в общем случае ограничивают интервалы бездействия между операциями чтения или записи, а не суммарную длительность запроса. Настройки подобраны для небольших учебных HTTP-запросов. В реальной системе значения проверяются под особенности клиентов, мобильных сетей и допустимых размеров загрузок.

Задание 4. Часть 2. CDN + WAF для L3/L4 DDoS

Защита от объемных DDoS-атак строится эшелонированно: внешний слой CDN или специализированный scrubbing-сервис поглощает сетевой поток и отбрасывает нежелательные пакеты (L3/L4). Затем WAF выполняет HTTP-анализ уровня L7, проверяет запросы на SQLi, XSS, сигнатуры сканеров и превышение частоты.



Рисунок 6 — архитектура «Пользователь → CDN (L3/L4) → WAF (L7) → Backend».

Доменное имя приложения указывает на CDN. На стороне origin-сервера прямой сетевой доступ следует ограничить доверенными исходящими IP-адресами CDN либо защищенным туннелем, иначе атакующий сможет обойти защиту и обращаться к backend напрямую.

На WAF запросы проверяются набором OWASP CRS и правилами, созданными оператором. После прохождения проверки Nginx передает легитимный запрос к backend Flask. CDN уменьшает объем трафика на origin, а WAF концентрируется на семантике HTTP и правилах защиты приложения.

Итоговая проверка соответствия методичке

Требование	Реализация	Проверка
1. SQLi: backend 200, WAF 403	curl и Docker Compose	Требует Docker/CRS
2. Правило User-Agent, ID 10010	nginx/custom.conf	Статика пройдена; Docker не запускался
3. 500×10 до ограничения	ApacheBench + локальный Nginx	Фактически: 500; 0 ошибок
3. 500×10 после ограничения	limit_req 20r/s, burst=30	Фактически: 469 Non-2xx; HTTP 503
4. Slowloris	4 директивы таймаутов	Конфигурация и пояснения
4. CDN + WAF	Схема и описание	Выполнено

Вывод: В ходе лабораторной работы подготовлены конфигурация межсетевого экрана OWASP CRS и пользовательское правило фильтрации User-Agent для варианта №10, настроено ограничение частоты запросов к /api/profile и разработан профиль защиты от Slowloris и объемных DDoS-атак. На системном Nginx успешно проверено поведение Rate Limiting: без ограничения все 500 запросов обработаны без ошибок, после включения ограничения 469 ответов оказались вне диапазона 2xx. Дополнительная проверка 100 параллельными запросами подтвердила 79 ответов HTTP 503.

Оставшаяся часть практического подтверждения — реальные коды 200/403 от Docker-контейнеров Flask и ModSecurity, а также запись журнала пользовательского правила ID 10010. Для этого в комплект включён проверочный PowerShell-скрипт, сохраняющий результаты в evidence/. До выполнения данной проверки работу WAF нельзя считать экспериментально подтвержденной.